Starling Data

An adaptive RAG compliance copilot — grounded answers, cited sources.

Purpose

Starling Data answers natural-language compliance questions for financial institutions by retrieving from a curated regulatory corpus and generating answers grounded in inline citations. The reference deployment is built around a representative broker-dealer / RIA ("Kestrel Securities, LLC"). The product is engineered around four principles:

- **Grounded answers** — every claim is backed by a retrieved source chunk; the model refuses rather than fabricates.
- **Auditability** — each response carries a trace of which chunks were retrieved and which were cited, so a compliance officer can verify the reasoning.
- **Evaluation discipline** — a benchmark suite measures retrieval precision and answer faithfulness; results are committed and regression-checked on every change.
- **Adaptive routing** — query complexity drives the retrieval strategy, so a simple lookup and a multi-document gap analysis don't pay the same cost.

The corpus mixes **external regulation** (SEC 15c3-1/15c3-3, Reg SHO, Rule 605/606; FINRA Rules 5310/3110/3130; Advisers Act §206, Marketing Rule, Custody Rule; FinCEN BSA/AML), **internal Kestrel policies** (best-ex, code of ethics, AML, market-access controls, supervision), and **enforcement examples** (FINRA AWC). See `project-goals.md` for the full scope.

Architecture

Stack

- **Next.js 16** (App Router) + **React 19** + **TypeScript** (strict)
- **LangGraph** for the retrieve → generate pipeline
- **Pinecone** for vector search
- **OpenAI** for embeddings (`text-embedding-3-small`) and generation (`gpt-4o-mini`)
- **Tailwind**, **Vitest**, **Playwright**, **SonarQube**

Folder layout

```
app/
├── page.tsx                Root → DashboardPage
├── layout.tsx              Root layout
├── Pages/                  Page-level components
│   ├── DashboardPage       Query input, answer, trace, citations panel
│   ├── EvaluationsPage     Benchmark runner + saved-report viewer
│   ├── LibraryPage         Browse corpus documents
│   ├── LibraryDetailPage   Single doc + chunk inspector
│   └── HistoryPage         Past queries
├── _components/            Reusable React components (answer card,
```

```
citations
│                           panel, metric card/tooltip, trace section, …)
├── _lib/corpus.ts          Corpus index + chunk fetch helpers
└── api/
    ├── query/              POST: invoke graph, return { answer,
citations, retrievals }
    ├── evaluations/
    │   ├── retrieval/      POST: pinpoint-precision on a benchmark item
    │   ├── answer/         POST: LLM-as-judge faithfulness scoring
    │   └── reports/        GET list / POST save / GET [id]
    └── test-pushover/      GET: diagnostic for cap notifications

pipeline/
├── graph.ts                StateGraph: START → retrieve → generate → END
├── state.ts                GraphStateAnnotation (query, retrievals,
answer, citations)
├── nodes/
│   ├── retrieve.ts         Pinecone searchRecords (top-5) → Retrieval[]
│   └── generate.ts         OpenAI call, inline [^N] citations, renumbered
first-seen
├── instrument.ts           Memory/CPU + build/query counters
├── daily-cap.ts            500/day request cap (UTC reset)
└── notify-pushover.ts      Fire-and-forget alert when cap is hit

ingest/
├── chunk.ts                Markdown chunker. Two modes:
│                             · Regulatory (SEC/FINRA/MSRB) — paragraph-
path aware
│                               (e.g. "(a)(2)(ii)") for pinpoint citations
│                             · Fallback (Kestrel/FinCEN) — heading-slug
addressing
│                           Target ~500 tokens with ~50-token overlap.
└── upsert.ts               Batch Pinecone upsert (100/req) with metadata
schema

corpus/                     ~35 markdown files with YAML front-matter
                             (title, authority, citation_id, jurisdiction,
doc_type,
                              effective_date, version_status, topic_tags,
source_url)

eval/
├── core.ts                 Benchmark schema (Zod) + metric computation
├── runner.ts               Eval loop: invoke graph per item, stream
results
├── benchmarks/pilot.yaml   10 hand-labeled items (query,
expected_chunk_ids,
                             keywords, archetype)
├── results/                Per-run reports (markdown + JSON, includes
corpus SHA
                             and model IDs for reproducibility)
└── saved-reports/          Committed baselines surfaced in the UI

shared/types.ts             Single source of truth: Citation, Retrieval,
```

```
Chunk,
                              ChunkMetadata, QueryResponse, GraphState

scripts/
├── ingest.ts                pnpm ingest      — corpus/*.md → chunks →
Pinecone
├── reset-index.ts           pnpm reset-index — clear Pinecone index (dev)
├── verify-eval.ts           pnpm verify-eval — validate expected_chunk_ids
after
                              corpus changes
└── sonar.cjs                pnpm sonar       — SonarQube scan
```

Pipeline flow

```
   ┌──────────┐   query   ┌──────────────┐   retrievals[]
┌──────────────┐
   │  client  │──────────►│  retrieve    │──────────────────►│  generate
│
   │ /api/query│          │  (Pinecone)  │                   │   (OpenAI)
│
   └──────────┘           └──────────────┘
└──────┬───────┘
                                                              │
                            { answer, citations, retrievals } ◄────┘
```

The graph is lazy-initialized on first query so build doesn't construct SDK clients. Citations are extracted as `[^N]` markers from the model output and renumbered by first-seen order.

Data ingestion

Run with `pnpm ingest`. The pipeline turns `corpus/*.md` into searchable vector records in Pinecone in five steps:

1. **Discover & parse.** Each markdown file is read with `gray-matter`, splitting YAML front-matter (`title`, `authority`, `citation_id`, `jurisdiction`, `doc_type`, `effective_date`, `source_url`, `version_status`, `topic_tags`) from the markdown body. Files missing required front-matter are skipped with a structured log line; unexpected errors fail the run fast.
2. **Chunk.** `chunkDocument` dispatches on `authority` and runs one of two modes:
   - **Regulatory mode** (SEC / FINRA / MSRB) — paragraph-marker-aware. The chunker tracks both a heading stack and a marker stack `(a)` → `(1)` → `(i)` (FINRA Supplementary Material `.01`–`.99`, MSRB `(a)` → `(i)`), emitting **one chunk per terminal paragraph** so citations can pinpoint the exact subsection. Ambiguous Roman markers like `(i)`, `(v)`, `(x)` are resolved against the current stack. Tiny header-only frames are merged into their child or dropped to keep retrieval signal clean.
   - **Fallback mode** (Kestrel / FinCEN / internal docs) — splits on H1/H2/H3 headers and packs sentences into chunks up to the token budget.
3. **Pack & overlap.** Both modes target ~500 tokens (~375 words) per chunk with ~50-token (~38-word) overlap between adjacent chunks. The sentence splitter never breaks mid-sentence or mid-

citation (e.g. `12 CFR 1026.18` stays intact).

4. **Assemble records.** Each chunk gets a deterministic ID — regulatory: `${citationId}::${paragraphPath}::p${N}`, fallback: `${citationId}::${slugifiedHeadingPath}` — and a flat metadata payload (no nested objects) carrying `heading_path`, `paragraph_path`, `version_status`, `topic_tags`, `effective_date`, etc. This rich metadata is what lets retrieval surface pinpoint citations and lets evaluation score answers against expected chunk IDs.
5. **Embed & upsert.** Chunks are batched in groups of 100 and sent to Pinecone via the integrated-embedding API (`index.upsertRecords`) — Pinecone embeds the `chunk_text` field server-side, so the ingest script never touches the embedding model directly.

Query path

The retrieve node hydrates `Retrieval[]` from Pinecone hits (top-5 by default); the generate node formats them into a numbered context block and prompts the LLM to either cite each claim with a `[^N]` marker or refuse the answer. Citations are extracted from the model output and renumbered by first-seen order before being returned to the client.

Evaluation

Compliance answers are only useful if they're correct, and "correct" in this domain means a specific subsection of a specific rule — not a paraphrase that's directionally right. Every change to the chunker, the prompt, the model, or the corpus is measured against a benchmark before it ships, so quality only moves forward.

Benchmark

Each item in `eval/benchmarks/pilot.yaml` is a hand-labeled question with the source chunk(s) it should retrieve, the load-bearing keywords a correct answer must convey, and a canonical reference answer drafted from those chunks. Items are tagged with a `category` (Direct fact, Spanning, Comparative, Numerical, Relationship, Holistic) so scores can be sliced by question type and weak archetypes spotted early.

After any corpus or chunker change, `pnpm verify-eval` confirms every `expected_chunk_id` still resolves to a real chunk — preventing silent drift between the benchmark and the index.

Two evaluation tracks

The Evaluations page (`/evaluations`) and the API behind it run two independent passes over the benchmark:

Retrieval quality — does the right source come back, and at what rank? For each item the system pulls the top-10 from Pinecone and computes:

- **MRR** (mean reciprocal rank) — the right source coming back somewhere in the top-10 isn't enough; we want it ranked first, because the LLM pays most attention to what sits at the top of its context. MRR is a score from 0 to 1 capturing how *early* the must-have content surfaces in the ranked list. For each load-bearing term from the cited source (e.g. `"quarterly"`, `"payment for`

`order flow"`) we find the rank of the first chunk containing it — position #1 scores 1.0, #2 scores 0.5, #3 scores 0.33 — and average across the question's terms.

- **nDCG@10** — MRR only credits the *first* hit per term, but ideally the whole top-10 should be useful: deeper relevant chunks give the LLM corroborating context, while irrelevant ones just distract. nDCG@10 is a quality score from 0 to 1 for the entire top-10 ranking, weighted so hits at the top count more than hits further down. We compute it by scoring each chunk for relevance (does it contain a load-bearing term?), discounting by position, and dividing by the score the *ideal* ordering would have produced — so the result reads as a percentage of perfect.
- **Keyword coverage** — before ranking matters, we have to check the LLM is even getting the right ingredients: if the load-bearing terms aren't anywhere in the retrieved context, no amount of clever generation will produce a correct answer. Keyword coverage is a score from 0 to 1 measuring the fraction of must-have terms that appear *somewhere* in the retrieved set, ignoring rank. Low coverage means the retriever isn't finding the right material at all; high coverage paired with a low MRR means it's finding the material but ranking it below distractors.

Answer quality — given the retrieved context, does the generated answer hold up? Each item runs the full pipeline and the resulting answer is graded by an LLM-as-judge (`gpt-4.1-nano` by default, overridable via `OPENAI_JUDGE_MODEL`) on three axes:

- **Accuracy** — does the answer match the reference, with the right rule cited?
- **Completeness** — does it cover the load-bearing facts the reference covers?
- **Relevance** — does it answer the question that was asked?

Roadmap

Done

- **M1 — Ingestion.** Corpus, chunker (regulatory + fallback modes), embeddings, Pinecone upsert.
- **M2 — RAG pipeline.** Two-node LangGraph (retrieve → generate) with grounded, cited answers.
- **M3 — UI.** Dashboard, library, library-detail, history, evaluations pages; trace view; citations panel.
- **Ops.** Daily request cap with Pushover alerts, lazy graph init, design-system component library.

In flight

- **Eval harness.** Pilot harness (10 items) shipping; UI surfaces saved reports with metric cards and tooltips.
- **UX polish.** Citations panel interactivity (click chip → open + scroll), portaled metric tooltips.

Planned

- **Production-grade evaluation.** Scale the benchmark to a broader set of hand-labeled questions, add faithfulness and refusal scorers, and gate every deploy on a CI regression check so quality only moves forward.
- **Adaptive query routing.** Classify each query and route it to the right retrieval strategy — fast path for simple lookups, multi-hop retrieval for cross-document analysis, explicit refusal when the corpus can't support an answer.
- **Cross-document gap analysis.** Compare an institution's internal policies against the external rules they're meant to implement, and flag the gaps.

- **Examination preparation.** Assemble the document packages a regulator would request, with human-in-the-loop review before anything leaves the building.
- **Regulatory change monitoring.** Web-augmented retrieval for time-sensitive queries, with proactive alerts when rules change.
- **Multi-turn conversations.** Carry context across follow-up questions so users can drill into a topic naturally.

Prerequisites

- Node.js 20+ (22 recommended)
- pnpm 10+ (`corepack enable && corepack prepare pnpm@latest --activate`)

Setup

```
pnpm install
cp .env.example .env.local   # then fill in credentials
```

Development

```
pnpm dev           # Next.js dev server at http://localhost:3000
pnpm ingest        # Load corpus → chunks → Pinecone
pnpm eval          # Run the pilot benchmark
pnpm verify-eval   # Validate benchmark chunk IDs after corpus changes

pnpm typecheck     # tsc --noEmit (strict)
pnpm lint          # eslint .
pnpm format        # prettier --write .
pnpm build         # production build
pnpm test          # vitest
pnpm test:e2e      # playwright (builds first)
```